



Chapter 7 – Advanced SQL

Part 3: PSM

Databases lectures

Dr Kai Höfig



SQL/PSM - Motivation

- ◆ **Problems** of client-server systems ("embedded SQL" and "CLI"):
 - Constant change of execution control between application (=client) and DBS (=server)
 - No optimisation across multiple statements possible

- ◆ **Solution:** **stored procedures**
 - Software modules administered & executed in DBMS (procedures/functions)
 - Called from applications and queries / action parts of triggers

- ◆ **Advantages** of stored procedures
 - Structuring tools for larger applications: redundancy-free representation of relevant aspects of the application functionality
 - Procedures only depend on the DBMS
 - Optimisation of procedures
 - Execution of the procedures under control of the DBMS
 - Rights assignment for procedures



SQL/PSM: the standard

- ◆ ANSI standard for procedural extensions: **SQL/PSM** (Persistent Stored Modules)
- ◆ **Components:**
 - Stored modules from procedures and functions
 - Individual routines
 - Integration of external routines (implemented in C, Java, . . .)
 - Syntax constructs for loops, conditions, etc.
- ◆ **Implementation (more or less compliant) in all current DBMS**
 - Oracle: PL/SQL
 - IBM DB2: very similar to SQL/PSM
 - Informix: SPL
 - Microsoft SQL Server: Transact-SQL
 - MySQL, PostgreSQL: similar to SQL/PSM



Example of PSM in a function

- ◆ Functions can be created and permanently stored in the database to reuse recurring calculations.

```
create function getAge(@product_id int)
returns int
begin
    declare @age int;

    select @age=YEAR(CURRENT_TIMESTAMP)-model_year
    from    products
    where   product_id=@product_id;

    return @age;
end;
```

- ◆ They can be used in queries. Other applications of PSM are procedures, trigger and scripts in general.

```
select *, dbo.getAge(product_id) from products
```



Variables

- ◆ Variables contain a single data value of a specific type
- ◆ Variable names must begin with an @ character
- ◆ Declaration of variables (with optional initialisation)

```
DECLARE @local_variable [AS] data_type [ = value ] [;]
```

- ◆ Assigning values to variables

```
SET @local_variable = expression [;]
```

Note: null is possible as a value for value and expression

- ◆ Example

```
DECLARE @i int = 0;  
SET @i = 10;
```



Assigning and printing variables in SELECT

- ◆ Variables can be assigned by means of "=" after SELECT

```
declare @number_of_products int;  
declare @avg_price decimal(8,2);  
select @number_of_products=COUNT(*), @avg_price=avg(list_price) from products;
```

- ◆ Also selects that return one attribute can be used to directly assign a value

```
set @avg_price = (select AVG(list_price) from products);
```

- ◆ If the SELECT returns multiple tuples, the values of the *last tuple* are used (but this is bad style)

```
declare @some_customer_id int;  
select @some_customer_id=customer_id from customers;
```

- ◆ Variables can be printed to console using

```
print 'Average price is ' + cast(@avg_price as varchar);
```



Control flow – If

- ◆ Statement blocks

```
BEGIN  
    { sql_statement | statement_block }  
END
```

- ◆ Conditional execution using if

```
IF Boolean_expression  
    { sql_statement | statement_block }  
[ ELSE  
    { sql_statement | statement_block } ];
```

- ◆ Conditional execution using case

```
CASE  
WHEN Boolean_expression THEN {result_expression}  
    [ ELSE {result_expression} ]  
END
```



Example for if and case

```
if @price<10
    set @price_category='A';
else if @price<100
    set @price_category='B';
else
    set @price_category='C';
```

```
set @price_category = case
    when @price<10 then 'A'
    when @price<100 then 'B'
    when @price<1000 then 'C'
end;
```




Loops

◆ WHILE loop

```
WHILE Boolean_expression  
  { sql_statement | statement_block | BREAK | CONTINUE }
```

BREAK

Terminates the **innermost** WHILE loop

CONTINUE

Restarts the WHILE loop (all statements after CONTINUE are ignored)



Functions

- ◆ A user-defined function is a Transact-SQL [...] routine that accepts parameters, performs an action, such as a complex calculation, and returns the result of that action as a value. **The database state may not be changed within a function and functions must be deterministic.**
- ◆ Creating a function (simplified)

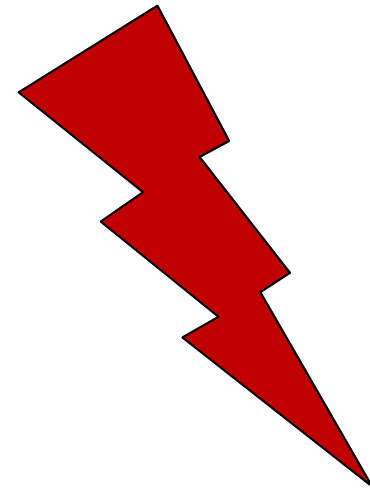
```
CREATE FUNCTION [ schema_name.] function_name
( [ { @parameter_name [ AS ] parameter_data_type  [ = default ] }
  [ ,...n ]
  ]
)
RETURNS return_data_type
[ AS ]
BEGIN
    function_body
    RETURN scalar_expression
END [ ; ]
```



Illegal operations in functions

- ◆ The database state may not be changed within a function.

```
create function not_allowed(@product_id int)
returns int
begin
    update products
    set list_price=100
    where product_id=@product_id;
    return 1;
end;
```



- ◆ Functions must be deterministic.

```
create function get_random_customer()
returns int
begin
    declare @rand_id int = rand()*(select COUNT(*) from customers);
    return @rand_id;
end;
```



Function example

```
create function price_category(@product_id int)
returns char
begin
declare @price_category char;
declare @list_price decimal(8,2) = (select list_price
                                     from products
                                     where product_id=@product_id);

if @list_price <100
    set @price_category='A';
else if @list_price<1000
    set @price_category='B';
else
    set @price_category='C';
return @price_category;
end;
```

◆ Use of this function in a select:

```
select product_name, list_price, dbo.price_category(product_id)
from products
where dbo.price_category(product_id)='A'
```



Procedures

- ◆ “Accept input parameters and return multiple values in the form of output parameters to the calling procedure or batch. Contain programming statements that **perform operations in the database**, including calling other procedures. Return a status value to a calling procedure or batch to indicate success or failure (and the reason for failure).” (Source: SQL Server Documentation)
- ◆ Creating a procedure (simplified)

```
CREATE { PROC | PROCEDURE } [schema_name.] procedure_name  
    [ { @parameter data_type } [ = default ] [ OUT | OUTPUT ] [ ,...n ]  
AS { [ BEGIN ]  
    sql_statement [;] [ ...n ]  
[ END ] } [;]
```

OUT | OUTPUT

Indicates that the parameter is an output parameter. Use OUTPUT parameters to return values to the caller of the procedure. (Source: SQL Server Documentation)



Procedures - example

- ◆ Procedure to increase prices by given percentage for a price category

```
create procedure increasePricesByCategory
@price_category char,
@percent int,
@number_of_prices_increased int out
as begin
update products
set list_price=list_price+(list_price*@percent)
where dbo.price_category(product_id)=@price_category;
end;
```

- ◆ Calling this procedure

```
declare @number_of_prices int;
execute dbo.increasePricesByCategory 'A', 10, @number_of_prices out;
print @number_of_prices;
```



Exercise: Strategy for ABC analysis using functions

- ♦ To ease the use of an ABC analysis, we need a function like

```
select customer_id, dbo.getCustomerClassification(customer_id) from customers
```

- ♦ To realize that function, it is helpful to create also functions that provide the sales value of a customer in total, the cumulated sales value of customers that spend more than a customer and the value ABC levels

```
getCustomerSales(id int)  
getCumulativeSales(id int)  
getABCLimit(level char)
```



Cursor – An iterator for results of selects

- ◆ A cursor is declared like a variable but instead of a value, it contains the result of a select

without the
leading @

```
declare product_cursor cursor for  
select product_name, list_price from products order by list_price asc;
```

- ◆ To use a cursor, we first have to open it, what triggers the generation of data

```
open product_cursor;
```

- ◆ After using it, we can throw the data away, since there is no garbage collector during the execution of e.g. a function

```
deallocate product_cursor;
```

<https://learn.microsoft.com/en-us/sql/t-sql/language-elements/declare-cursor-transact-sql?view=sql-server-ver16>



Cursor – Iterating through the results

```
declare product_cursor cursor for  
select product_name, list_price from products order by list_price asc;
```

1
Declare

```
open product_cursor;
```

2
Open

```
declare @a_product_name varchar(255)='';  
declare @a_list_price decimal(10,2)=0.0;
```

3
Fetch first tuple into
local variables

```
fetch next from product_cursor INTO @a_product_name, @a_list_price;
```

```
while @@FETCH_STATUS = 0  
begin
```

4
..is 0 as long as there are
remaining tuples in the cursor

```
    print @a_product_name+' costs '+cast(@a_list_price as varchar);  
    fetch next from product_cursor INTO @a_product_name, @a_list_price;  
end;
```

5
fetch again while looping

```
deallocate product_cursor;
```

6
Deallocate

```
FETCH [ [ NEXT | PRIOR | FIRST | LAST |  
        ABSOLUTE { n | @nvar } | RELATIVE { n | @nvar } ]  
FROM ] cursor_name } INTO @variable_name [ ,...n ]
```



Trigger

- ◆ Trigger are PSM scrips that are executed when data in tables changes. They are registered for a certain table and a certain event.
- ◆ Applications
 - Enforcing integrity constraints ("implementation" of integrity rules)
 - Auditing of DB actions
 - Propagation of DB changes
- ◆ Specification of
 - Event and condition for activating the trigger
 - Action(s) to execute
 - This is why it is often called the ECA (event-condition-action) rule
- ◆ Available in most commercial systems (different syntax)

<https://learn.microsoft.com/de-de/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-ver16>



Example: Trigger for delivery dates before order dates

- ◆ We want every new order to have a delivery date in the future. Otherwise, we do not insert the data.

```
create trigger DeliveryInFuture on orders
instead of insert as
begin
    if not exists (select * from inserted where required_date < order_date)
        insert into orders (customer_id, order_status, order_date,
                           required_date, shipped_date, store_id, staff_id )
        select  customer_id, order_status, order_date, required_date,
                shipped_date, store_id, staff_id
        from    inserted;
end;
```

- ◆ How can we modify the trigger, so that the required date is at least the delivery date?



Trigger entry point

- ◆ Trigger can be registered to execute when an insert, update or delete operation is executed.
- ◆ Trigger can **replace** the regular execution of insert, update and delete operations using the keyword **instead of**. The affected tuples are not part of the database yet. Not possible for views with check option.
- ◆ Trigger can **complement the successful** execution of insert, update and delete operations using the keyword **after**. The affected tuples are part of the database and all operations are completed (e.g. on delete operations, constraints etc.). Not possible for views.



The `inserted` table

- ◆ During the execution of a trigger, the table `inserted` exists. This table has the same schema as the table on which the trigger is registered for. It contains the tuples the trigger was executed for.
- ◆ The `inserted` table contains for the following operations and entry points..

	<code>insert</code>	<code>update</code>	<code>delete</code>
<code>instead of</code>	the tuples that are part of the insert operation that caused the trigger to execute. The tuples are not part of the database yet.	the tuples that are about to be inserted (<code>update=delete+insert</code>) into the database. The tuples are not part of the database yet.	no tuples
<code>after</code>	the tuples that are part of the insert operation that caused the trigger to execute. The tuples are copies of the tuples that are already part of the database.	the new tuples that were changed during the update operation. The tuples are copies of the tuples that are already part of the database.	no tuples

- ◆ This table can contain more than one tuple, for example if one insert operation contains multiple tuples.



The deleted table

- ◆ During the execution of a trigger, the table `deleted` exists. This table has the same schema as the table on which the trigger is registered for. It contains the tuples the trigger was executed for.
- ◆ The `deleted` table contains for the following operations and entry points..

	<code>insert</code>	<code>update</code>	<code>delete</code>
<code>instead of</code>	no tuples	the tuples that are about to be deleted (update=delete+insert) from the database. The tuples are copies and still part of the database.	the tuples that are about to be deleted (update=delete+insert) from the database. The tuples are copies and still part of the database.
<code>after</code>	no tuples	the old tuples that were changed during the update operation. The tuples are already deleted from the database.	the tuples that were deleted. The tuples are already deleted from the database.

- ◆ This table can contain more than one tuple, for example if one delete operation contains multiple tuples.



rollback

- ◆ Sometimes it makes sense to undo an operation. The command `rollback` reverts to the beginning of a transaction.
- ◆ In combination with a trigger, the transaction consist of the commands to be executed including trigger operations. Executing a rollback during a trigger reverts the operations the trigger was executed for. This only makes sense for after trigger, since in an instead of trigger, nothing has happened yet.

```
create trigger no_negative_prices on products
after insert as
begin
    if exists (select * from inserted where list_price<0) rollback;
end;
```

- ◆ A rollback causes an exception reported to the caller.

The transaction ended in the trigger. The batch has been aborted.